An Example of Operator Overloading

Function overloading in C++:

You can have multiple definitions for the same function name in the same scope. The definition of the function must differ from each other by the types and/or the number of arguments in the argument list. You can not overload function declarations that differ only by return type.

Following is the example where same function **print()** is being used to print different data types:

```
#include <iostream>
using namespace std;

class printData
{
   public:
      void print(int i) {
        cout << "Printing int: " << i << endl;
      }

      void print(double  f) {
        cout << "Printing float: " << f << endl;
      }

      void print(char* c) {
        cout << "Printing character: " << c << endl;
      }
};

int main(void)
{
   printData pd;

   // Call print to print integer
   pd.print(5);
   // Call print to print float
   pd.print(500.263);
   // Call print to print character
   pd.print("Hello C++");

   return 0;
}
```

When the above code is compiled and executed, it produces the following result:

```
Printing int: 5
Printing float: 500.263
Printing character: Hello C++
```

Operators overloading in C++:

You can redefine or overload most of the built-in operators available in C++. Thus a programmer can use operators with user-defined types as well.

Overloaded operators are functions with special names the keyword operator followed by the symbol for the operator being defined. Like any other function, an overloaded operator has a return type and a parameter list.

```
Box operator+(const Box&);
```

declares the addition operator that can be used to **add** two Box objects and returns final Box object. Most overloaded operators may be defined as ordinary non-member functions or as class member functions. In case we define above function as non-member function of a class then we would have to pass two arguments for each operand as follows:

```
Box operator+(const Box&, const Box&);
```

Following is the example to show the concept of operator over loading using a member function. Here an object is passed as an argument whose properties will be accessed using this object, the object which will call this operator can be accessed using **this** operator as explained below:

```
#include <iostream>
using namespace std;

class Box
{
   public:

      double getVolume(void)
      {
         return length * breadth * height;
      }
      void setLength( double len )
      {
          length = len;
      }

      void setBreadth( double bre )
      {
          breadth = bre;
      }

      void setHeight( double hei )
      {
          height = hei;
      }
      // Overload + operator to add two Box objects.
      Box operator+(const Box& b)
      {
         Box box;
         box.length = this->length + b.length;
         box.breadth = this->breadth + b.breadth;
         box.height = this->height + b.height;
         return box;
      }
   private:
      double length;      // Length of a box
      double breadth;     // Breadth of a box
      double height;      // Height of a box
};
// Main function for the program
int main( )
{
   Box Box1;                // Declare Box1 of type Box
   Box Box2;                // Declare Box2 of type Box
   Box Box3;                // Declare Box3 of type Box
   double volume = 0.0;     // Store the volume of a box here

   // box 1 specification
   Box1.setLength(6.0);
   Box1.setBreadth(7.0);
   Box1.setHeight(5.0);

   // box 2 specification
   Box2.setLength(12.0);
   Box2.setBreadth(13.0);
   Box2.setHeight(10.0);

   // volume of box 1
```

```
   volume = Box1.getVolume();
   cout << "Volume of Box1 : " << volume <<endl;

   // volume of box 2
   volume = Box2.getVolume();
   cout << "Volume of Box2 : " << volume <<endl;

   // Add two object as follows:
   Box3 = Box1 + Box2;

   // volume of box 3
   volume = Box3.getVolume();
   cout << "Volume of Box3 : " << volume <<endl;

   return 0;
}
```

When the above code is compiled and executed, it produces the following result:

```
Volume of Box1 : 210
Volume of Box2 : 1560
Volume of Box3 : 5400
```

Overloading the Increment/Decrement Operators

This explains how to overload the increment and decrement operators. The example involves only the increment operator, but the decrement operator is handled in exactly the same way.

Consider the following definition of a simple Clock class:

```
class Clock {
    friend ostream& operator<<(ostream& out, Clock& c);
  public:
    Clock(int h=12, int m=0);    // noon by default
    Clock operator++();                 // prefix form; advance clock by 1
min
    Clock operator++(int);       // postfix form; advance clock by 1 min
  private:
    void tick();                 // advance clock by 1 min
    int hour, min;
};
```

We want to be able to use Clock objects like this:

```
#include <iostream>

int main()
{
  Clock c1, c2(3,20), c3(10,59);

  cout << c1 << " " << c2 << " " << c3 << endl;
  // should output 12:00 03:20 10:59
```

```
  c1++;
  ++c2;
  c3++;

  cout << c1 << " " << c2 << " " << c3 << endl;
  // should output 12:01 03:21 11:00

  c1 = ++c2;
  cout << c1 << " " << c2 << endl;
  // should output 03:22 03:22

  c1 = c3++;
  cout << c1 << " " << c3 << endl;
  // should output 11:00 11:01

  return 0;
}
```

The constructor and tick() methods are easy:

```
Clock::Clock(int h, int m) : hour(h), min(m) { }

// Advance the object's min by 1
void Clock::tick()
{
  min = (min + 1) % 60;
  if (min == 0)
    hour = (hour + 1) % 24;
}
```

The leftshift operator is also easy:

```
ostream& operator<<(ostream& os, Clock& c)
{
  if (c.hour < 10)               // then pad the hour with a 0
    os << '0';
  os << c.hour << ":";
  if (c.min < 10)                // then pad the min with a 0
    os << '0';
  os << c.min;
  return os;
}
```

Ok, now the prefix form of the increment operator. For syntax like:

```
c1 = ++c2;
```

we need the c2 object to increment itself first, and then for the resulting object to be assigned to c1. Remember, the righthand side of the assignment is equivalent to c2.operator++().

```
Clock Clock::operator++()    // prefix form
{
  tick();             // increment
  return *this;       // return itself for assignment
}
```

Now for the postfix version, i.e., syntax like:

```
c1 = c3++;
```

The compiler needs some way to distinguish this method from the prefix form. C++ makes this distinction by requiring the postfix increment operator to take an int argument. There is absolutely no significance to this argument, and in fact it does not even need a name. Think of the "int" as a meaningless symbol that says to the compiler "I am the postfix version of the increment operator." The righthand side of the preceding assignment is equivalent to c3.operator++(int).

```
Clock Clock::operator++(int)    // parameter ignored and does
{                               // not even need to be named.
  Clock c = *this;  // Save the object.
  tick();           // Increment the object.
  return c;         // Return the object as it was before incrementing.
}
```